

NAME	PAGES	STREAK/CLASS	DATE - TIME
Joshua David Sierra J.	1 / 10	Prog. para Mecatronica	24-5-2026
Title: Capitulo IV		Topic: Fundamentos axiomáticos y Proposiciones	

Keyword	Notes
Proposición Atómica	La lógica matemática es la disciplina formal
Lógica matemática	que estudia los métodos de razonamiento, suministrando
Proposición Molecular	reglas universales para determinar la validez de un
Valor de verdad	argumento.

Una proposición es un enunciado matemático o lógico que solo puede ser verdadero (1) o falso (0). Por ejemplo:

• Ejemplo de hardware (si es proposición):

"El pin digital D2 del microcontrolador está en alto (5V)."

• Ejemplo en texto (no es proposición):

"Apaga el motor." (Esto es una orden no se puede evaluar como 1 o 0).

Las proposiciones atómicas son aquellas que cuentan con solo una condición y luego están las compuestas con dos o más.

Questions and Reflections

¿Por qué una estructura de control condicional `if()` en lenguaje C exige estrictamente que su argumento sea una proposición? Porque el microcontrolador evalúa las condiciones basándose en si el resultado es cero o diferente de cero (1).

Summary: Una proposición en mecatrónica usando lenguaje C es cualquier evento o lectura que el procesador evalúa como 1 o 0. Si carece de conectores lógicos es una proposición atómica (simple) y si se combinan varios condiciones mediante operadores lógicos se transforman en una compuesta o molecular.

NAME: Joshua David Sierra S. PAGES: 2 / 10 SPEAKER/CLASS: Prog. para mecatrónica DATE-TIME: 29 - 5 - 2026

Title: Capítulo IV Topic: Conectores lógicos y operadores en C

Keyword	Notes
Conjunción (&)	Conjunción ($\wedge$) $\rightarrow$ Operador & (&)
Disyunción ()	Con este podremos hacer que de 1 solo si todas las
Negación (!)	entradas son 1. Es decir deben cumplirse todas las condiciones.
Condicional (if)	Disyunción ($\vee$) $\rightarrow$ operador ()
	Con este daría 1 si al menos una entrada es 1. Es decir con que se cumpla una condición es suficiente.
	Negación ($\sim$) $\rightarrow$ Operador ! (NOT)
	Bloquea el comando cuando el valor con el signo ! está presente en el código.
	Condicional (if)
	Tiene que cumplirse una condición para que ocurra una acción. Si ocurre un antecedente, se ejecuta el consecuente.

Questions and Reflections

¿Qué error puede ocurrir en una máquina si confundo el operador & con el operador || en el código de seguridad? No funcionaría de forma correcta. Con & el robot solo arranca si la puerta está cerrada y el operario llega. Con || el robot podría encenderse con la puerta abierta y el operario lejos, causando un accidente.

Summary: Los conectivos lógicos controlan el flujo en C de forma exacta: la conjunción ($\wedge$) que funciona si todo se cumple, la disyunción que simula si se cumple una condición ($\vee$). Operadores como estos crean condiciones lógicas para el control seguro de actuadores mecánicos.

NAME
Joshua David Sierra J.

PAGES
3 / 10

SPEAKER/CLASS
Programación para Mec.

DATE - TIME
29-5-2026

Title:
Capítulo IV

Topic:
Tablas de Verdad

Keyword

Notes

Tabla de Verdad Es un gráfico matemático que muestra todos los
Combinaciones (2^n) resultados posibles de una expresión lógica según sus
Tautología (1) entradas.

Contradicción (0)

Contingencia Fórmula de filas (2^n)

Número de filas = 2

Número de variables = n

Evaluemos la expresión

$$[(p \rightarrow q) \wedge \sim q] \rightarrow \sim p$$

Tenemos 2 variables (p, q) $\rightarrow 2^2 = 4 \rightarrow$ cantidad de filas

Tabla de verdad evaluada:

P	q	$P \rightarrow q$	$\sim q$	$(P \rightarrow q) \wedge \sim q$	$\sim p$
1	1	1	0	0	0
1	0	0	1	0	0
0	1	1	0	0	1
0	1	1	1	1	1

Questions and Reflections

¿Para qué le sirve a un programador mecatrónico conocer la tabla de verdad?
Permite simplificar condicionales gigantescos o usar múltiples conectores lógicos,
esto nos ayuda a ahorrar memoria RAM y Flash en el microcontrolador.

Summary: Una tabla de verdad analiza el comportamiento de un
sistema lógico binario. Con esto comprobamos si el código tiene una
tautología (siempre da 1), una contradicción (siempre da 0) o una contingencia
(depende de los señales de los sensores).

NAME	PAGES	SPEAKER/CLASS	DATE - TIME
Joshua David Sierra J.	4 / 10	Prog. para Microcontroladores	29 - 5 - 2020
Title: Capítulo IV		Topic: Inferencia lógica y reglas de deducción	

Keyword	Notes
Inferencia Lógica	Proceso en el cual se obtienen conclusiones válidas a partir de un conjunto de proposiciones verdaderas llamados premisas.
Premisas (H_n)	
Conclusión (C)	
Modus Ponens	Reglas fundamentales (modelos en C)
Modus Tollens	A) Modus Ponendo Ponens (M.P.P.):
Silogismo	"Afirmando, afirmo".
	• Ecuación: $[(P \rightarrow Q) \wedge P] \rightarrow Q$
	B) Modus Tollendo Tollens
	"Negando, niego"
	• Ecuación: $[(P \rightarrow Q) \wedge \sim Q] \rightarrow \sim P$

Questions and Reflections

¿Cómo aplica un microcontrolador las reglas de inferencia lógica de forma automática al ejecutar un programa? Lo hace a través de su unidad de control y el Program Counter. Cuando el código evalúa una premisa en un if, genera un salto condicional directo hacia la subrutina que representa la conclusión.

Summary: La inferencia lógica nos permite deducir nuevos estados del sistema en C sin necesidad de agregar hardware redundante. Utilizando reglas como el Modus Ponens o el Modus Tollens.

NAME	PAGES	SPEAKER/CLASS	DATE - TIME
Joshua David Sierra J.	5 / 10	Prog. para Mecatronica	29-5-2026
Title: Capitulo IV		Topic: Lógica de predicados y cuantificadores	

Keyword	Notes
Predicado $P(x)$	Un predicado es un enunciado que contiene variables libres y se convierte en proposición cuando se le asigna un valor específico a dicha variable.
Argumento / Variable	
Cuantificador Universal ($\forall$)	
Cuantificador Existencial ($\exists$)	
Universo de Discurso	Un cuantificador Universal ($\forall$) establece que el predicado es verdadero para todos los valores del universo mencionado, del sistema.
Bucle for/while	
	Un cuantificador existencial ($\exists$) $\rightarrow$ indica que el predicado es verdadero para al menos un valor del universo.
	Algunos ejemplos serían: para el universal, validar que todos los motores de una máquina estén apagados para darle mantenimiento. Por otro lado, el existencial equivale a que se dispare una alarma si un sensor registra algo.

Questions and Reflections

¿Cómo se interpreta el concepto de "Universo de discurso" al programar la lectura de un sensor analógico en C? El universo de discurso es el rango de valores válidos que puede tomar la variable. Para un convertidor analógico-digital de 10 bits del microcontrolador, este es estrictamente el conjunto de números enteros $[0, 1023]$.

Summary: La lógica de predicados permite parametrizar condiciones lógicas mediante funciones en C que reciben variables. El cuantificador Universal exige que todos los elementos cumplan la condición, mientras que el existencial busca que al menos un elemento cumpla el predicado.

NAME	PAGES	SPEAKER/CLASS	DATE-TIME
Joshua David Silva J.	8 / 10	Prac. para matemáticas	21-5-2024
Title: Capítulo IV		Topic: Principios de Demostración formal: Método directo	

Keyword	Notes
Demostración Método directo Hipótesis Conclusión Condicional	Este es un proceso lógico y estructurado para comprobar la validez de un teorema o proposición condicional ($P \rightarrow Q$) asumiendo verdades previas. Método directo Se asume que la hipótesis o premisa inicial (H) es verdadera, (1) y se aplican los reglas algebraicas e inferencias paso a paso para deducir directamente que la conclusión también es verdadera. Aplican en programación Demostramos que si el entero x es par, se puede escribir matemáticamente como $x = 2K$ (donde K es un entero). Elevamos ambos al cuadrado: $x^2 = (2K)^2 = 4K^2$. Ahora factorizamos: $x^2 = 2 \cdot (2K^2)$. Como x^2 es de la forma $2 \cdot (\text{entero})$, queda demostrado que x^2 es par.

Questions and Reflections

¿Por qué el método de demostración directo es el más eficiente al estructurar rutinas lógicas críticas en el firmware de un PLC? Porque al seguir una secuencia lineal y deductiva, el procesador ejecuta las instrucciones de forma predecible y rápida.

Summary: El método directo de demostración consiste en partir de una hipótesis verdadera para llegar de forma lineal a una conclusión válida mediante pasos lógicos encadenados.

NAME: Joshua David Sierra J. PAGES: 7 / 10 SPEAKER/CLASS: Prog. para mecatrónica DATE-TIME: 29-5-2024

Title: Capítulo IV Topic: Demostración por contradicción

Keyword	Notes
Contradicción	Para demostrar que una proposición P es verdadera,
Suposición inversa	se asume al comienzo que P es falsa ($\sim P$). Si al
Falsedad (0)	desarrollar la lógica se llega a una inconsistencia
Depuración	matemática o absurdo (0), queda demostrado que la
Absurdo lógico	suposición inicial era errónea y, por lo tanto, P es
Morep de error	obligatoriamente verdadera.

Ejemplo:

Si queremos demostrar por contradicción hay que hacer lo siguiente: En este caso tendremos que si x^2 es un número impar, entonces x es impar.

1- Supongamos lo contrario a la conclusión: Supongamos que x es par

2- Si x es par, entonces: $x^2 = (2K)^2 = 4K^2 = 2(2K^2)$

3- Esto contradice al enunciado que dice que x^2 es impar.

Questions and Reflections

¿Cómo utiliza un programador mecatrónico el razonamiento por contradicción al buscar un cortocircuito o un error en el código? Asumiendo que el componente que cree dañado funciona perfectamente. Si al medir con el multímetro el Voltaje de 0V donde debería haber 5V, demuestra que ese componente o código está dañado.

Summary: La demostración por contradicción valida una propuesta demostrando que su alternativa inversa es lógicamente imposible. Este enfoque es la base para diseñar el sistema de detección de errores y rutinas de contingencia en C.

NAME	PAGES	SPEAKER/CLASS	DATE - TIME
Joshua David Sierra J.	8 / 10	Prog. para Microelectrónica	24-5-2024
Title: Capítulo IV		Topic: Principio de Inducción Matemática	

Keyword	Notes
Inducción	Método con el que podemos demostrar que la fórmula o una propiedad se cumple para todos los números enteros de forma infinita. Funciona como una especie de efecto dominó y cuenta con dos pasos fundamentales: el primero, demostrar que la fórmula funciona para el primer elemento ($n=1$). El segundo es demostrar que si la propiedad se cumple para un número cualquiera K , entonces se cumple obligatoriamente para el número siguiente $K+1$.
Caso base ($n=1$)	
Hipótesis inductiva	
Paso inductivo	
Bucles infinitos	

Questions and Reflections

¿Qué pasa en un bucle while en C si se le olvida programar el "paso inductivo" (el incremento del contador)? El bucle se vuelve infinito y el programa del microcontrolador se bloquea. Al no existir el paso inductivo, la variable nunca cambia de valor.

Summary: El principio de inducción matemática demuestra que un proceso funciona al validar un caso base ($n=1$) y su continuidad sucesiva ($K+1$).

NAME: Joshua David Sierra J. PAGES: 4 / 10 SPEAKER/CLASS: Prog. para Mecatronica DATE - TIME: 29-5-2026

Title: Capitulo IV Topic: Equivalencias de razonamiento

Keyword	Notes
Equivalencia Lógica vs C Diagnóstico Flujo de control Sensores	Para entender este capitulo, se resume como cada concepto abstracto del libro se conecta directamente con una línea de código real:
Concepto del libro	Equivalente en código C
Proposición	Variable bool (0 o 1)
	Aplicación en la Mec. Estado de un sensor (detecta / no detecta).
Conjunción (∧)	Operador &&
	Por condiciones obligatorias.
Disyunción (∨)	Operador
	Una condición es suficiente.
Condicional (→)	if
	Monitores de rones de un sensor ondulante.
Predicado	función con argumento
	Escaneo de los sensores de la maquina continuamente

Questions and Reflections

¿Cuál es la ventaja de ver la programación en C como si fuera lógica matemática? Nos permite revisar y corregir los errores del programa en papel antes de cargar el código al microcontrolador.

Summary: Este capitulo demuestra que la programación y la lógica matemática funcionan igual. Un sensor es una proposición, los condicionales son conectivos lógicos y los bucles son procesos inductivos.

NAME	PAGES	SPEAKER/CLASS	DATE - TIME
Joshua David Sierra J.	10 / 10	Prog. para Mecatrónicos	24-5-2024

Title:

Capítulo IV

Topic:

Álgebra de Boole

Keyword

Álgebra de Boole
Compuertas lógicas
Leyes de Morgan
Simplificación
Esquemáticos

Notes

Es un sistema matemático que simplifica expresiones lógicas binarias. Se utiliza tanto para reducir el número de compuertas físicas en un circuito eléctrico como para limpiar líneas de código redundantes en C.

Leyes de De Morgan

Estas permiten cambiar el operador principal de un condicional negando los términos internos:

1- $!(A \& B)$ es exactamente igual a $!A || !B$

2- $!(A || B)$ es exactamente igual a $!A \& !B$

Questions and Reflections

¿Para qué le sirve a un programador mecatrónico simplificar una expresión lógica usando el Álgebra de Boole? Le sirve para que el microcontrolador ejecute el programa mucho más rápido y gaste menos energía.

Summary: Las leyes de De Morgan y el Álgebra de Boole permiten transformar y reducir expresiones lógicas complejas en el Código C.

NAME	PAGES	SPEAKER/CLASS	DATE - TIME
Joshua David Sierra J.	2 / 10	Prog. para Mecatrónica	29-5-2026

Title: Capítulo V

Topic: Sistemas de numeración y arquitectura

Keyword	Notes
Conversión	Para convertir de decimal a binario (Método Manual)
División Sucesiva	se divide el número decimal entre 2 sucesivamente.
Agrupación de Bits	Los residuos de abajo hacia arriba forman el número
Desplazamiento	binario.
Depuración	Ejemplo: Convertir 13 a binario $13 \div 2 = 6$ (sobra 1) $\rightarrow$ Bit menos significativo (derecha) $6 \div 2 = 3$ (no sobra nada) $3 \div 2 = 1$ (sobra 1) $1 \div 2 = 0$ (sobra 1) $\rightarrow$ Bit más significativo (izquierda) $13 = 0b1101$ Para convertir de binario a hexadecimal se separa el binario en grupos de 4 bits y se traduce cada grupo.

Questions and Reflections

¿Qué pasa si un programa en C envía el número decimal 10 a un chip que espera recibir los datos estrictamente en código binario puro? El chip receptor interpretará el número como el binario. Esto activaría solo la primera salida, el hardware encenderá ambos pines, provocando un error en la máquina.

Summary: En base a lo hablado, las conversiones permiten traducir la información entre el formato humano, el formato de desarrollo y el estado real del hardware.

NAME: Joshua David Sierra D. PAGES: 3 / 10 SPEAKER/CLASS: Prog. para Mecatrónicos DATE-TIME: 29-5-2020

Title:

Capítulo V

Topic:

Representación de números negativos

Keyword

Notes

Bit de signo	Los microcontroladores no entienden el símbolo de menos (-). Solo procesan cables con voltaje (1) o sin voltaje (0).
Magnitud	
Complemento a 1	Para solucionar esto, se usa el bit de signo: el bit más a la izquierda define el signo. Si el número es cero positivo; si es 1 el número es negativo.
Complemento a 2	
Desbordamiento	
Enteros (int)	Para obtener el complemento a 2 es el método matemático estándar que usan los procesadores para guardar números negativos en las variables tipo int.

Questions and Reflections

¿Por qué es útil el complemento a 2 al leer un sensor de temperatura industrial que mide valores bajo cero? Porque permite que el programa reciba y procese temperaturas bajo cero de forma automática.

Summary: El complemento a 2 es el sistema matemático que permite representar números negativos en binario sin usar símbolos físicos de resta.

NAME: Joshua David Sierra J. PAGES: 4 / 10 SPEAKER/CLASS: Prog. para Mecatrónica DATE - TIME: 29-5-2026

Title:

Capítulo V

Topic:

Operadores a nivel de bits en C

Keyword

Notes

Bitwise A diferencia de los operadores lógicos normales (&&, ||), AND a nivel de bits los operadores bitwise modifican los bits de una OR a nivel de bit (|) variable uno por uno de forma individual. Son la XOR a nivel de bit (^) herramienta principal para encender y apagar pines del microcontrolador.

Operador AND

Compara bit a bit. Da 1 solo si ambos bits son 1. Se usa para apagar bits específicos sin alterar los demás.

Operador OR

Compara bit a bit. Da 1 si al menos uno de los bits es 1. Se usa para encender pines específicos.

Questions and Reflections

¿Por qué no se debe usar el operador lógico & normal cuando quieres apagar un pin físico del microcontrolador? Porque el &, terminaría borrando o alterando la configuración de todos los pines relacionados del puerto al mismo tiempo.

Summary: Los operadores a niveles de bits modifican de manera individual cada uno de los cables lógicos dentro de un registro en C. Son indispensables en mecatrónica para crear "máscaras de bits", lo que permite encender, apagar o alternar estados lógicos de la máquina.

NAME
Joshua David Sierra J.

PAGES
5 / 10

SPEAKER/CLASS
Prog. para Mecatronica

DATE - TIME
29-5-2020

Title:

Capítulo 5

Topic:

Bit Shifting

Keyword

Desplazamiento
shift left
Shift right
Multiplicación
Binaria

Notes

Es una operación que mueve todos los bits de una variable hacia la izquierda o hacia la derecha, rellenando los espacios vacíos con ceros.
Es el método más rápido que tiene el hardware para hacer multiplicaciones y divisiones.

Tipos de desplazamiento en C

A- Desplazamiento a la izquierda ($\ll$)

Mueve los bits hacia la izquierda. Cada posición desplazada equivale a multiplicar el número por 2.

B- Desplazamiento a la derecha

Se mueven los bits hacia la derecha. Cada posición desplazada equivale a dividir el número entre 2.

Questions and Reflections

¿Por qué un programador mecatrónico usa $\ll$ para multiplicar en lugar del operador de multiplicación normal? Porque es mucho más rápido tener que realizar varios ciclos de sumas internas, mientras que el desplazamiento de bits se procesa en un solo ciclo de reloj del microcontrolador.

Summary: Los operadores de desplazamiento ($\ll$ y $\gg$) mueven la estructura binaria de una variable posiciones a la izquierda o derecha.

NAME

Joshua David Sierra J.

PAGES

6 / 10

SPEAKER/CLASS

Prog. para Microcontroladores 24-5-2026

DATE - TIME

Title:

Capítulo V

Topic:

Máscaras de bits

Keyword

Notes

Máscaras
 Limpieza de bits
 Modificación segura

Una máscara de bits son un valor binario hecho para aislar, encender o apagar bits determinados.

Para controlar un puerto físico, combinamos los máscaras con las operaciones vistas en la página anterior.

Máscara para encender

Usamos un mapa donde solo el bit deseado es 1.

Máscara para apagar

Creamos una máscara invertida, donde el bit a borrar o mejor dicho, el bit a apagar es 0 y todos los demás son 1.

Questions and Reflections

¿Por qué es peligroso modificar un registro completo asignándole un valor en lugar de una máscara? Porque al hacer una asignación directa borramos los estados de todos los demás pines del puerto. Al escribir el valor directo, apagamos funciones críticas.

Summary: Las máscaras de bits son configuraciones de bits (binarios) que sirven para modificar pines específicos de un microcontrolador sin alterar el estado de componentes vecinos.

NAME
Joshua David Sierra J.

PAGES
7 / 10

SPEAKER/CLASS
Prog. para mecatrónica 29-5-2020

DATE - TIME

Title:
Capítulo V

Topic:
Registros de configuración GPIO

Keyword	Notes
GPIO	Para controlar cualquier pin físico del microcontrolador en C, el hardware utiliza un trío de registros de 8 bits. Cada bit de estos registros se conecta a un pin del chip.
DDR (dirección)	
Port (escritura)	
Pin (lectura)	
Entrada/salida	
	1- Registro DDR (Data Direction Register): Configura el pin como autopista de entrada o salida.
	2- Registro PORT (Data Register): Escribe el estado físico del pin (envía energía).
	3- Registro PIN (Input Pins Address): Lee el voltaje que entra al chip desde el exterior.

Questions and Reflections

¿Qué pasa si intentas encender un motor usando PORT sin haber configurado el registro DDR antes? El motor no avanzará. Si el registro DDR se queda en cero, el pin estará en alta impedancia (desconectado por dentro).

Summary: Los pines GPIO se controlan con tres registros: DDR define si el pin es entrada o salida, PORT inyecta o extrae el voltaje de salida, y PIN lee el voltaje que llega desde un sensor externo.

Joshua David Sierra S.

8 / 10

Prog. para Mecatrónica

29-5-2026

Title:

Capítulo V

Topic:

Memoria Caché y velocidad de ejecución

Keyword

Notes

Memoria caché
SRAM vs DRAM
Latencia
Hit / Miss
Jerarquía
Registro CPU

La memoria ultra rápida de tamaño muy pequeño ubicada dentro de la misma CPU se llama memoria caché. Esta guarda copia de los datos de la RAM que el código usa más seguido para no perder tiempo buscándolos fuera.

La cache SRAM esta hecha de transistores. No necesita refrescarse, es cara pero trabaja a la velocidad del procesador.

La DRAM hecha de capacitores. Necesita refrescarse constantemente, es mas lenta pero almacena gigabytes de datos.

Cache Hit (Acerto): El dato que necesita el if o bucle de C ya está en la caché. Se lee al instante.

Cache miss (Fallo): El dato no está. La CPU tiene que detenerse

Questions and Reflections

¿Cómo afecta la jerarquía de memoria (desde los registros hasta la RAM) el diseño de un algoritmo de control en tiempo real en sistemas mecatrónicos? Un algoritmo en tiempo real debe responder siempre en el mismo intervalo de tiempo; si el código esta desordenado, provocará constantes "Cache Miss".

Summary: La memoria caché (SRAM) es un puente de alta velocidad entre los registros de la CPU y la Ram principal. Evita que el procesador pierda tiempo esperando datos externos.

Joshua David Sierra J.

4 / 10

Prog. para Mecatronica

29-5-2026

Title:

Capitulo V

Topic:

Pipelining

Keyword

Notes

Pipelining

Ciclo de reloj

Fetch (Búsqueda)

Decode

Execute (Ejecución)

Hazard (Peligro)

Es una técnica que consiste en dividir la ejecución de instrucciones en pasos secuenciales para procesar varios órdenes al mismo tiempo.

Etapas básicas del procesador

1- Fetch (Búsqueda): Trae la instrucción desde la memoria.

2- Decode (Decodificación): La CPU interpreta qué debe hacer.

3- Execute (Ejecución): Se realiza la operación.

Si el código tiene un if o un salto inesperado, la cadena de montaje se rompe. La CPU debe vaciar el pipeline y empezar de nuevo, perdiendo ciclos de reloj.

Questions and Reflections

¿Por qué el abuso de condicionales if/else en C afecta el rendimiento del procesador a nivel de hardware? Cada vez que la CPU no sabe qué camino tomará el if, se arriesga a precalcular instrucciones incorrectas. Si falla, tiene que reiniciar el proceso, ralentizando el proceso.

Summary: El pipelining acelera el procesador ejecutando distintos etapas de varias instrucciones a la vez. Evitar saltos lógicos en C que sean innecesarios. Garantizando que el microcontrolador ejecute las tareas a su máxima velocidad.

NAME
Joshua David Sierra J.

PAGES
10 / 10

SPEAKER/CLASS
Prog. para Mecatrónica

DATE - TIME
29-5-2026

Title:
Capítulo V

Topic:
Endianness

Keyword

Notes

Endianness	Es el orden en el que se guardan los bytes de los
Big-Endian	números grandes en un microcontrolador dentro de los
Little-Endian	casillos de la memoria RAM.
Byte más significativo	
Byte menos significativo	Hay dos métodos de organización:
Memoria RAM	Imaginemos que tenemos el número en hexadecimal de 32 bits: 0x12345678. Este número se divide en 4 bytes: 12, 34, 56 y 78.
	A) Big-Endian (orden natural): Guarda el byte más importante (12) siendo la dirección de memoria más baja el primer byte.
	B) Little-Endian (orden inverso): Guarda el byte menos importante (78) en la dirección de memoria más baja (primero). En este caso el orden sería [78][56][34][12]

Questions and Reflections

¿Qué problema ocurre si un sensor industrial transmite datos en formato Big-Endian pero tu microcontrolador los lee en Little-Endian? Ocurre un error de interpretación de datos grave. La máquina interpretará una posición totalmente descontrolada y peligrosa debido al choque de arquitecturas.

Summary: El Endianness define la orientación y el orden en el que los bytes se almacenan en la memoria RAM. Es importante conocer esta diferencia arquitectónica al programar la comunicación entre sensores.